

数据层文档

作者: 李彦锴 yankail520@gmail.com

目录

- 1. 原理
 - 1.1 抓取引擎 DataEngine
 - 1.2 数据中心 DataCenter
 - 1.3 特征 Feature
- 2. DataCenter 使用示例
- 3. 脚本编写示例
 - 3.1 fetch 数据源脚本
 - 3.2 Feature 特征脚本
 - 3.3 Object 数据对象脚本
- 4. 字段说明
- 5. 分层: raw 层与 feature 层

1. 原理

整个数据层由三个组件构成，均采用插件式架构，遵循"约定优于配置"原则：各组件扫描约定目录，通过动态导入与反射发现基类的子类，自动完成注册并统一调度，新增数据源、数据对象或特征时只需在对应目录下新建文件。契约层由抽象基类定义，具体行为采用模板方法模式，由子类实现 `fetch()`、`to_daily()`、`compute()` 等步骤。DataCenter 以注册表的方式维护表名到数据源、数据对象类的映射，作为全部模块读取数据的统一入口，并采用懒加载与缓存；数据对象的 `filter()` 借鉴了 Django ORM 的字段查询语法（`字段__操作符=值`）。特征之间的依赖关系构成一个有向无环图（DAG），以类继承表达，采用协作式多继承约定，由 Python 的方法解析顺序（MRO，C3 线性化）决定执行顺序；每个叶子类独立执行其完整依赖链，共享祖先在各条链中分别计算，引擎通过同名列取值一致性校验完成合并去重。

组件	扫描目录	发现的基类	职责
DataEngine	<code>co_data_pipeline/scripts/</code>	<code>DataSourceBase</code>	调度原始数据的抓取与落盘
DataCenter	<code>co_data_pipeline/scripts/</code> 、 <code>co_features/sources/</code> 、 <code>co_data_center/objects/</code>	<code>DataSourceBase</code> 、 <code>DataObjectBase</code>	统一的数据读取入口
FeatureEngine	<code>co_features/features/</code>	<code>FeatureBase</code>	构建宽表并计算特征

1.1 抓取引擎 DataEngine

代码位置: `co_data_pipeline/engine.py` 运行方式: `python -m co_data_pipeline.engine`

数据源基类 DataSourceBase

每个数据源是 `DataSourceBase` (`co_data_pipeline/pipeline_base.py`) 的一个子类, 必须实现以下六个抽象方法:

方法	作用
<code>fetch(**params) -> DataFrame</code>	从外部接口获取数据
<code>get_fields() -> dict</code>	声明字段名及类型
<code>get_table_name() -> str</code>	声明表名, 也是落盘的文件名
<code>save(df) -> None</code>	将数据写入磁盘
<code>load() -> DataFrame</code>	从磁盘读取数据
<code>is_calendar_anchor() -> bool</code>	声明是否为时间轴基准表

说明: `is_calendar_anchor()` 目前是基类中的声明, 调度代码中没有读取它; 宽表的时间轴基准由 `FeatureEngine` 中的常量 `_BASE_TABLE = "raw_stock_prices"` 确定。

清洗装饰器 data_clear

`common/decorators.py` 中的 `data_clear(clear_func)` 是一个装饰器工厂, 作用是让 `fetch()` 的返回值在返回前自动经过 `clear_func` 处理:

```
@data_clear(_clean_stock_prices)
def fetch(self, **params) -> pd.DataFrame:
    ...
    return raw_df          # 实际返回的是 _clean_stock_prices(raw_df)
```

每个数据源传入各自专属的清洗函数, 签名为 `(df: DataFrame) -> DataFrame`。

运行流程

1. **加载环境变量:** 导入模块时, 通过 `python-dotenv` 读取项目根目录的 `.env` 文件 (例如 `ALPHA_VANTAGE_KEY`) 。
2. **发现数据源:** `find_scripts()` 按文件名排序扫描 `co_data_pipeline/scripts/*.py` (跳过以 `_` 开头的文件), 以 `co_data_pipeline.scripts.<文件名>` 为模块名动态导入, 找出其中定义的 `DataSourceBase` 具体子类, 并以无参数方式实例化。
3. **读取启用开关:** 读取 `configs/enabled_sources.json`, 键为表名, 值为是否启用。配置中没有出现的表视为未启用; 配置文件不存在或解析失败时, 视为空配置, 不运行任何数据源。
4. **逐个执行:** 对每个启用的数据源依次执行:
 - `fetch()`: 不传任何参数, 因此使用各数据源的默认参数;

- 字段校验：比较返回的列名与 `get_fields()` 的声明，缺少或多出字段只打警告，不中断；
 - `save(df)`。
5. **失败隔离**：任一数据源的 `fetch()` 或 `save()` 抛出异常，只将该数据源记为失败，继续执行其他数据源。
 6. **汇总**：最后输出成功与失败的表名列表。

当前的启用配置（`configs/enabled_sources.json`）：

```
{
  "raw_stock_prices": true,
  "raw_vix": true,
  "raw_treasury_rates": true,
  "raw_news": true,
  "raw_option_chain": false,
  "raw_dividends": true
}
```

DataEngine 只扫描 `co_data_pipeline/scripts/`，不会运行 `co_features/sources/` 下的 `FeatureDailySource`。

1.2 数据中心 DataCenter

代码位置：`co_data_center/data_center.py`

DataCenter 是所有模块读取数据的统一入口。任何模块需要数据时，都应通过 `DataCenter.get()` 获取，而不是直接读取文件。

初始化时做的两件事

1. 发现数据源，建立 {表名：数据源实例} 映射

依次扫描两个目录：

- `co_data_pipeline/scripts/`：原始数据源
- `co_features/sources/`：内部计算结果的 `DataSourceBase` 包装（目前是 `FeatureDailySource`，对应表 `feature_daily`）

发现机制与 DataEngine 相同。如果不同目录中出现同名表，以最后扫描到的为准，并打印警告。

2. 发现数据对象类，建立 {表名：数据对象类} 映射

扫描 `co_data_center/objects/`，找出 `DataObjectBase` 的子类（排除 `DataObject` 和 `DataObjectBase` 本身），依据类属性 `SOURCE_TABLE` 确定它对应哪张表。未声明 `SOURCE_TABLE` 的类会被跳过并打印警告。

懒加载与缓存

- 初始化时**不读取任何数据**。
- 第一次调用 `get(table_name)` 时，才执行该数据源的 `load()` 读取文件，并用对应的数据对象类包装；没有特化数据对象类的表，用通用的 `DataObject` 包装。
- 包装后的对象存入缓存，之后对同一张表的 `get()` 直接返回缓存中的同一个对象。
- 缓存属于 DataCenter 实例，不同的 DataCenter 实例之间不共享缓存。

数据对象的分层

数据对象定义在 `co_data_center/data_center_base.py`，分为三层：

类	性质	内容
<code>DataObjectAbstract</code>	抽象，纯基类	持有 <code>DataFrame</code> (<code>df</code> 属性)，声明抽象方法 <code>to_daily()</code>
<code>DataObjectBase</code>	抽象	提供通用操作 <code>filter()</code> 和 <code>merge()</code> ， <code>to_daily()</code> 仍未实现
<code>DataObject</code>	具体	<code>to_daily()</code> 默认恒等返回，适用于本身就是一天一行的表

需要特殊日频处理的表，继承 `DataObject` 并覆写 `to_daily()`，放在 `co_data_center/objects/` 下。目前有三个特化类：

类	对应表	<code>to_daily()</code> 的行为
<code>DividendObject</code>	<code>raw_dividends</code>	将 <code>ex_date</code> 重命名为 <code>date</code> ，只返回 <code>date</code> 、 <code>amount</code> 两列
<code>NewsObject</code>	<code>raw_news</code>	按日期聚合：情绪分按 <code>relevance_score</code> 加权平均，并统计当日新闻篇数
<code>OptionChainObject</code>	<code>raw_option_chain</code>	直接抛出 <code>NotImplementedError</code> ，要求改用 <code>filter(date=...)</code> 取某个快照日的完整期权链

`df` 属性以只读方式暴露底层 `DataFrame`，外部不应直接修改。

1.3 特征 Feature

代码位置：`co_features/feature_base.py`、`co_features/feature_engine.py` 运行方式：`python -m co_features.feature_engine`

特征基类 `FeatureBase`

每个特征是 `FeatureBase` 的一个子类，必须实现两个抽象方法：

方法	作用
<code>compute(df) -> DataFrame</code>	在输入 <code>DataFrame</code> 上追加本特征的列并返回
<code>get_feature_name() -> str</code>	本特征对应的列名

两个关键机制：

实例化即计算：`FeatureBase.__init__` 被装饰器 `apply_computation` 装饰。实例化时先保存传入的 `DataFrame`，然后自动调用 `self.compute()`，结果通过 `.df` 属性取出：

```
result_df = DailyReturnFeature(base_df).df
```

用继承表达依赖：如果特征 B 依赖特征 A 的计算结果，就让 B 继承 A，并在 B 的 `compute()` 中先调用 `super().compute(df)`，再追加自己的列。这样实例化 B 时，A 的计算会沿继承链自动先完成。`FeatureBase.compute()` 本身原样返回 `df`，是继承链的终点。例如 `RollingVolatility20DFeature` 和 `VixCorrelationFeature` 都继承 `DailyReturnFeature`，因为它们依赖 `daily_return` 列。

FeatureEngine 的运行流程

第一步：构建基础宽表 (`_build_base()`)

1. 以 `raw_stock_prices` 的 `to_daily()` 结果为基础，按日期排序，并校验 `date` 列存在、无缺失、无重复。
2. 按 `dc.list_tables()` 的顺序遍历其余各表，决定是否并入：
 - 读取数据对象上的 `include_in_wide_table` 属性，若有声明则以它为准；
 - 若未声明，则 `_DEFAULT_EXCLUDED_TABLES` 中的表（`raw_stock_prices`、`raw_option_chain`、`feature_daily`）不并入，其余并入。目前所有数据对象类都没有声明该属性。
3. 对要并入的表调用 `to_daily()`；若抛出 `NotImplementedError`，则跳过。
4. 按 `date` 做左连接（`how="left"`，`validate="one_to_one"`），合并前后做以下校验：
 - 除 `date` 外存在同名列：报错，要求在该表的 `to_daily()` 中改名；
 - `date` 存在缺失或重复：报错；
 - 合并后行数发生变化：报错。

由于是以股价表为左表的左连接，宽表的行就是 `raw_stock_prices` 中的交易日；其他表中不在这些日期上的行不会进入宽表。

第二步：发现特征类

用 `importlib.import_module` 按文件名顺序导入 `co_features/features/*.py`（跳过以 `_` 开头的文件，该目录必须有 `__init__.py`），收集其中定义的 `FeatureBase` 具体子类，**排除抽象类**。

第三步：筛选叶子类

在已发现的类中，没有被其他任何类继承过的类是“叶子类”。只实例化叶子类，祖先类的计算通过继承链自动完成，不单独实例化。

第四步：计算并合并

对每个叶子类：

1. 用基础宽表的副本实例化，取得结果；
2. 校验结果的 `date` 列，以及行数与基础宽表一致；
3. 结果中与已有列同名的列（来自共享祖先，例如多个叶子都算出了 `daily_return`），要求取值与已有值一致（数值列容差 `1e-12`），不一致则报错；
4. 只把新增的列按 `date` 合并进结果。

第五步：失败处理

`FeatureEngine(strict=True)`（默认）时，任何特征文件导入失败或特征类计算失败，都会在最后抛出 `RuntimeError`；`strict=False` 时只记录错误并跳出。

第六步：落盘

`run()` 调用 `build()` 得到结果，再通过 `FeatureDailySource().save()` 写入 `data/features/feature_daily.parquet`。之后即可通过 `DataCenter.get("feature_daily")` 读取。

边界约定

FeatureEngine 只负责"无状态、只看过去"的特征（`shift`、`diff`、`rolling` 等）。需要拟合参数的处理（缩尾阈值、标准化参数等）不放在这里，应在时间序列切分之后只用训练集拟合。

2. DataCenter 使用示例

2.1 创建与查看

```
from co_data_center.data_center import DataCenter

dc = DataCenter()
print(dc)
# <DataCenter tables=['raw_dividends', 'raw_news', 'raw_option_chain',
#                      'raw_treasury_rates', 'raw_stock_prices', 'raw_vix', 'feature_daily']>

dc.list_tables()          # 列出所有表名，不触发加载
```

表名的顺序即扫描顺序：先是 `co_data_pipeline/scripts/` 下按文件名排序的各数据源，然后是 `co_features/sources/`。

2.2 获取数据

```
stock = dc.get("raw_stock_prices")    # 首次调用时读取文件
type(stock).__name__                  # 'DataObject'（没有特化类的表）
stock.df                              # 底层 DataFrame（只读使用）

news = dc.get("raw_news")
type(news).__name__                   # 'NewsObject'

dc.get("raw_vix") is dc.get("raw_vix") # True: 第二次直接命中缓存
```

2.3 条件筛选 `filter()`

参数格式为 `字段名__操作符=值`，不写操作符时默认为 `eq`。多个条件之间为"且"的关系。

操作符	含义
<code>eq</code>	等于（默认）
<code>ne</code>	不等于
<code>gt</code> / <code>gte</code>	大于 / 大于等于
<code>lt</code> / <code>lte</code>	小于 / 小于等于
<code>in</code>	属于给定列表

操作符	含义
<code>contains</code>	字符串包含（先将字段转为字符串再匹配）

```
# 日期区间
recent = stock.filter(date__gte="2018-01-01", date__lte="2024-12-31")

# 链式调用
high_price = stock.filter(date__gte="2018-01-01").filter(close__gt=100)

# 字符串包含、属于列表
news.filter(title__contains="dividend")
news.filter(source__in=["Reuters", "Bloomberg"])

# 期权链：取某个快照日，再看看涨期权
chain = dc.get("raw_option_chain")
snapshot = chain.filter(date="2024-01-02")
calls = snapshot.filter(option_type="call")
```

`filter()` 不修改原对象，返回同一类型的新对象（例如对 `NewsObject` 筛选后仍是 `NewsObject`），结果的索引会被重置。

2.4 转为日频 `to_daily()`

```
news.to_daily()           # 列: date, sentiment_score, news_count
dc.get("raw_dividends").to_daily() # 列: date, amount
dc.get("raw_vix").to_daily()      # DataObject: 原样返回

chain.to_daily()           # 抛出 NotImplementedError
```

2.5 合并 `merge()`

```
merged = dc.get("raw_stock_prices").merge(dc.get("raw_vix"), on="date", how="left")
type(merged).__name__           # 'DataObject'
```

`merge()` 的参数为另一个数据对象、连接键 `on` 和连接方式 `how`（默认 `"left"`），结果统一包装为通用的 `DataObject`。

2.6 读取特征宽表

```
fd = dc.get("feature_daily").filter(date__gte="2018-01-01")
fd.df[["date", "close", "rate_1y", "dividend_yield", "rolling_vol_20d"]]
```

2.7 常见错误

情况	异常
<code>dc.get()</code> 的表名不存在	<code>KeyError</code> ，提示可用表名
<code>filter()</code> 使用了不支持的操作符	<code>ValueError</code>
<code>filter()</code> 使用了不存在的字段	<code>KeyError</code> ，提示可用字段
对 <code>raw_option_chain</code> 调用 <code>to_daily()</code>	<code>NotImplementedError</code>
表对应的 <code>parquet</code> 文件尚未生成	由 <code>load()</code> 中的 <code>pd.read_parquet</code> 抛出文件不存在的异常

3. 脚本编写示例

以下示例中的表名、类名均为示意，编写时替换为实际内容即可。

3.1 fetch 数据源脚本

放在： `co_data_pipeline/scripts/`（文件名不以 `_` 开头） **启用：** 在 `configs/enabled_sources.json` 中加入 `"<表名>": true`

以从 FRED 读取一个利率序列为例，结构与 `fetch_retes.py` 一致：

```
import pandas as pd

from co_data_pipeline.pipeline_base import DataSourceBase
from common.decorators import data_clear
from common.paths import get_raw_path
from common.logger import get_logger

lg = get_logger("co_data_pipeline")

def _clean_example(df: pd.DataFrame) -> pd.DataFrame:
    """本数据源专属的清洗逻辑，fetch() 的返回值会自动经过这里。"""
    df = df.copy()
    df["date"] = pd.to_datetime(df["date"])
    return df.sort_values("date").reset_index(drop=True)

class ExampleRateSource(DataSourceBase):
    """示例：3 个月期美国国债利率（FRED 代码 DGS3MO）。"""

    SERIES_ID = "DGS3MO"
    START_DATE = "2018-01-01"

    @data_clear(_clean_example)
    def fetch(self, **params) -> pd.DataFrame:
        import pandas_datareader.data as web # 延迟导入
```



```

series_id = params.get("series_id", self.SERIES_ID)
start = params.get("start_date", self.START_DATE)

raw = web.DataReader(series_id, "fred", start=start).reset_index()
raw.columns = ["date", "rate_3m"]
return raw[list(self.get_fields())]

def get_fields(self) -> dict:
    return {"date": "datetime64[ns]", "rate_3m": "float64"}

def get_table_name(self) -> str:
    return "raw_example_rates"

def is_calendar_anchor(self) -> bool:
    return False

def save(self, df: pd.DataFrame) -> None:
    path = get_raw_path() / f"{self.get_table_name()}.parquet"
    path.parent.mkdir(parents=True, exist_ok=True)
    df.to_parquet(path, index=False)
    lg.info(f"{self.get_table_name()}: 已写入 {path}, 共 {len(df)} 行")

def load(self) -> pd.DataFrame:
    path = get_raw_path() / f"{self.get_table_name()}.parquet"
    return pd.read_parquet(path)

```

编写要点：

- `fetch()` 由 DataEngine 以无参数方式调用，所有参数都应通过 `params.get(键, 默认值)` 提供默认值。
- 字段名要与 `get_fields()` 的声明一致，否则 DataEngine 会打出警告。
- 日期列统一命名为 `date`。若主键日期列叫其他名字（如 `raw_dividends` 的 `ex_date`），需要配一个数据对象类，在 `to_daily()` 中改名（见 3.3）。
- 若要并入宽表，除 `date` 外的列名不能与其他表重复，否则 FeatureEngine 会报错。
- `save()` 的写入策略由数据源自行决定。现有脚本中，`raw_stock_prices` 为全量覆盖写入，其余表为按主键去重的增量追加（新数据覆盖同主键的旧行，其余保留）。

完成后，这张表会同时被 DataEngine（负责抓取）和 DataCenter（负责读取）自动发现，并在下次运行 FeatureEngine 时并入宽表。

3.2 Feature 特征脚本

放在：`co_features/features/`（文件名不以 `_` 开头）

示例一：只依赖基础宽表中的列

```

import pandas as pd

from co_features.feature_base import FeatureBase

```

```

class Rate3mMomentumFeature(FeatureBase):
    """示例：3 个月期利率的 20 日变化量。"""

    WINDOW = 20

    def compute(self, df: pd.DataFrame) -> pd.DataFrame:
        df = super().compute(df)  # 必须无条件先调用
        df = df.copy()
        df["rate_3m_momentum_20d"] = df["rate_3m"].diff(self.WINDOW)
        return df

    def get_feature_name(self) -> str:
        return "rate_3m_momentum_20d"

```

示例二：依赖另一个特征的结果

依赖 `daily_return`，就继承 `DailyReturnFeature`：

```

import numpy as np
import pandas as pd

from co_features.features.daily_return import DailyReturnFeature

class RollingVolatility60DFeature(DailyReturnFeature):
    """示例：60 日滚动年化波动率。"""

    WINDOW = 60
    TRADING_DAYS_PER_YEAR = 252

    def compute(self, df: pd.DataFrame) -> pd.DataFrame:
        df = super().compute(df)  # 先由父类算出 daily_return
        df = df.copy()
        df["rolling_vol_60d"] = (
            df["daily_return"].rolling(window=self.WINDOW).std()
            * np.sqrt(self.TRADING_DAYS_PER_YEAR)
        )
        return df

    def get_feature_name(self) -> str:
        return "rolling_vol_60d"

```

编写要点：

- `compute()` 中必须**无条件先调用** `super().compute(df)`，保证继承链上的计算完整执行。
- `compute()` 和 `get_feature_name()` 都必须实现，否则该类是抽象类，会被 `FeatureEngine` 排除。
- 只使用当日及以前的数据（`shift`、`diff`、`rolling` 等），不能增删行。
- 新列名不能与已有列重名；若与共享祖先产出的列同名，取值必须一致。
- 若 A 已被 B 继承，`FeatureEngine` 只会实例化 B，A 的列由 B 的继承链产出。

3.3 Object 数据对象脚本

放在：co_data_center/objects/（文件名不以 _ 开头）

当一张表不是"一天一行、日期列叫 date"的结构时，需要为它编写数据对象类，在 to_daily() 中转换。以 DividendObject 为例：

```
import pandas as pd

from co_data_center.data_center_base import DataObject

class DividendObject(DataObject):
    SOURCE_TABLE = "raw_dividends" # 必须声明，对应 get_table_name() 的返回值

    def to_daily(self) -> pd.DataFrame:
        df = self._df.copy()
        df = df.rename(columns={"ex_date": "date"})
        return df[["date", "amount"]]
```

需要按天聚合的表，参照 NewsObject：在 to_daily() 中按日期 groupby 聚合，返回一天一行的 DataFrame。

语义上不适合转为日频的表，参照 OptionChainObject：在 to_daily() 中直接抛出 NotImplementedError，FeatureEngine 遇到时会跳过该表。

编写要点：

- 必须声明类属性 SOURCE_TABLE，否则 DataCenter 会跳过该类。
- 继承 DataObject，即可直接获得 filter()、merge()。
- to_daily() 的返回结果中，date 不能有缺失或重复，否则并入宽表时会报错。
- 本身已是一天一行、日期列叫 date 的表，不需要编写数据对象类，DataCenter 会自动用 DataObject 包装。

4. 字段说明

4.1 raw_stock_prices

- 数据源：Alpha Vantage TIME_SERIES_DAILY_ADJUSTED（fetch_stock.py）
- 文件：data/raw/raw_stock_prices.parquet
- 写入方式：全量覆盖
- 数据对象类：DataObject
- 角色：宽表的时间轴基准（is_calendar_anchor() 返回 True）

字段	类型	含义
date	datetime64	交易日期
open	float64	前复权开盘价：open_raw × close / close_raw

字段	类型	含义
high	float64	前复权最高价: $\text{high_raw} \times \text{close} / \text{close_raw}$
low	float64	前复权最低价: $\text{low_raw} \times \text{close} / \text{close_raw}$
close	float64	前复权收盘价 (接口 5. adjusted close), 按拆股和分红调整
volume	int64	按拆股调整后的成交量: volume_raw 乘以该日之后所有拆股系数的乘积
open_raw	float64	未调整的原始开盘价 (接口 1. open)
high_raw	float64	未调整的原始最高价 (接口 2. high)
low_raw	float64	未调整的原始最低价 (接口 3. low)
close_raw	float64	未调整的原始收盘价 (接口 4. close)
volume_raw	int64	未调整的原始成交量 (接口 6. volume)

口径说明:

- 前复权以最新价格为基准向过去调整, 每次新的分红或拆股都会使全部历史复权值整体变化。
- 与未调整的分红金额配合计算时 (如股息率), 必须使用 `close_raw`。
- 接口的拆股系数 (`8. split coefficient`) 只在清洗时用于调整成交量, 不保存。

4.2 raw_dividends

- 数据源:** Alpha Vantage `DIVIDENDS` (`fetch_dividends.py`)
- 文件:** `data/raw/raw_dividends.parquet`
- 写入方式:** 按 `ex_date` 去重的增量追加
- 数据对象类:** `DividendObject`

字段	类型	含义
ex_date	datetime64	除息日, 本表主键
declaration_date	datetime64	宣布日
record_date	datetime64	登记日
payment_date	datetime64	支付日
amount	float64	每股分红金额 (未调整)

日期字段无法解析时为缺失值。 `to_daily()` 返回 `date` (即 `ex_date`) 和 `amount` 两列。

4.3 raw_news

- **数据源**: Alpha Vantage `NEWS_SENTIMENT` (`fetch_news.py`) , 默认起始日期 `2018-01-01` , 按月分段请求
- **文件**: `data/raw/raw_news.parquet`
- **写入方式**: 按 `date` + `url` 去重的增量追加
- **数据对象类**: `NewsObject`

字段	类型	含义
<code>date</code>	<code>datetime64</code>	发布时间 (含时分秒, 接口 <code>time_published</code>)
<code>title</code>	<code>object</code>	新闻标题
<code>source</code>	<code>object</code>	新闻来源
<code>url</code>	<code>object</code>	新闻链接
<code>relevance_score</code>	<code>float64</code>	文章与 JPM 的相关度 (接口 <code>ticker_sentiment</code> 中 JPM 对应的 <code>relevance_score</code>)
<code>sentiment_score</code>	<code>float64</code>	针对 JPM 的情绪分, 由接口的 <code>ticker_sentiment_score</code> 从 [-1, 1] 线性映射到 [0, 1]: $(x + 1) / 2$

情绪分取自文章中 JPM 对应的 `ticker_sentiment` 片段, 而非整篇文章的整体情绪; 文章中找不到 JPM 时, 这两个分数为缺失值。

`to_daily()` 的输出:

字段	含义
<code>date</code>	发布时间去掉时分秒后的日期
<code>sentiment_score</code>	当日各篇新闻情绪分的加权平均, 权重为 <code>relevance_score</code> ; 当日权重之和为 0 时, 取简单平均
<code>news_count</code>	当日新闻篇数

4.4 raw_treasury_rates

- **数据源**: FRED `DGS1` , 经 `pandas_datareader` 读取 (`fetch_retes.py`) , 默认起始日期 `2018-01-01`
- **文件**: `data/raw/raw_treasury_rates.parquet`
- **写入方式**: 按 `date` 去重的增量追加
- **数据对象类**: `DataObject`

字段	类型	含义
<code>date</code>	<code>datetime64</code>	日期

字段	类型	含义
rate_1y	float64	1 年期美国国债收益率，百分比数值（FRED 原始单位）

清洗时对 `rate_1y` 先前向填充、再后向填充缺失值。

4.5 raw_vix

- **数据源**：FRED `VIXCLS`，经 `pandas_datareader` 读取（`fetch_vix.py`），默认起始日期 `2018-01-01`
- **文件**：`data/raw/raw_vix.parquet`
- **写入方式**：按 `date` 去重的增量追加
- **数据对象类**：`DataObject`

字段	类型	含义
date	datetime64	日期
vix	float64	CBOE 波动率指数日收盘值

清洗时对 `vix` 先前向填充、再后向填充缺失值。

4.6 raw_option_chain

- **数据源**：Alpha Vantage `HISTORICAL_OPTIONS`（`fetch_option.py`）
- **快照日期**：默认 `CALIBRATION_DATES`，2018-2024 年每年年初一个交易日
- **文件**：`data/raw/raw_option_chain.parquet`
- **写入方式**：按 `date` + `expiration_date` + `strike` + `option_type` 去重的增量追加
- **数据对象类**：`OptionChainObject`（不支持 `to_daily()`）
- **启用状态**：在 `configs/enabled_sources.json` 中为 `false`
- **用途**：Heston 模型参数校准，不并入宽表

字段	类型	含义
date	datetime64	快照日期（接口缺少该字段时，用请求的快照日期代替）
underlying	object	标的代码（接口 <code>symbol</code> ）
expiration_date	datetime64	到期日（接口 <code>expiration</code> ）
strike	float64	行权价
option_type	object	期权类型（接口 <code>type</code> ）
bid	float64	买价
ask	float64	卖价
last_price	float64	最新成交价（接口 <code>last</code> ）

字段	类型	含义
<code>volume</code>	float64	成交量
<code>open_interest</code>	float64	持仓量
<code>implied_volatility</code>	float64	隐含波动率
<code>delta</code>	float64	Delta
<code>gamma</code>	float64	Gamma
<code>theta</code>	float64	Theta
<code>vega</code>	float64	Vega
<code>rho</code>	float64	Rho

4.7 feature_daily

- **生成方式**: `FeatureEngine` (`co_features/feature_engine.py`)
- **文件**: `data/features/feature_daily.parquet`
- **数据源包装**: `FeatureDailySource` (`co_features/sources/feature_daily_source.py`) , 其 `fetch()` 调用 `FeatureEngine().build()`
- **行**: `raw_stock_prices` 的全部交易日

基础宽表部分 (按并入顺序) :

字段	来源	说明
<code>date ...</code> <code>volume_raw</code>	<code>raw_stock_prices</code>	共 11 列, 含义见 4.1
<code>amount</code>	<code>raw_dividends</code>	仅在除息日有值, 其余日期为缺失
<code>sentiment_score</code>	<code>raw_news</code> (<code>NewsObject.to_daily()</code>)	当日没有新闻时为缺失
<code>news_count</code>	<code>raw_news</code> (<code>NewsObject.to_daily()</code>)	当日没有新闻时为缺失
<code>rate_1y</code>	<code>raw_treasury_rates</code>	数据源默认起始日期 2018-01-01, 之前为缺失
<code>vix</code>	<code>raw_vix</code>	数据源默认起始日期 2018-01-01, 之前为缺失

由于是左连接, `raw_news` 等表中不在股价交易日上的日期 (如周末) 不会进入宽表。

特征部分 (`co_features/features/`) :

raw 层

- **内容**: 从外部接口获取、经各数据源专属清洗函数处理后的原始数据，每个数据源一张表，保留各自的原始结构（例如新闻为逐条记录、期权链为一天多行、分红以除息日为主键）。
- **生产者**: DataEngine 调度 `co_data_pipeline/scripts/` 下的各数据源。
- **路径**: `common/paths.py` 中的 `get_raw_path()`，即 `data/raw/`。
- **职责边界**: 只做与单个数据源相关的处理（字段映射、类型转换、日期解析、复权、缺失值填充等），不涉及跨表的合并或特征计算。

feature 层

- **内容**: `feature_daily` 一张宽表，以 `raw_stock_prices` 的交易日为时间轴，一天一行，包含并入的各 raw 表日频字段和计算出的特征。
- **生产者**: FeatureEngine。
- **路径**: `common/paths.py` 中的 `get_features_path()`，即 `data/features/`。
- **职责边界**: 负责跨表对齐与特征计算，只包含"无状态、只看过去"的特征。

两层之间的衔接

- **DataCenter 是两层共用的读取入口**: `co_data_pipeline/scripts/` 下的数据源对应 raw 层，`co_features/sources/` 下的 `FeatureDailySource` 对应 feature 层，两者以同样的方式被 DataCenter 发现和读取，下游模块不需要关心数据来自外部接口还是内部计算。
- **数据对象的 `to_daily()` 是 raw 层结构进入 feature 层的转换点**: 它负责把各表转为"一天一行、日期列为 `date`"的统一形式；不适合转换的表（`raw_option_chain`）不进入 feature 层，下游直接从 raw 层按快照日期读取。
- **两层分别由不同入口生成**: raw 层由 `python -m co_data_pipeline.engine` 更新，feature 层由 `python -m co_features.feature_engine` 更新；DataEngine 不会运行 `FeatureDailySource`。raw 层更新后，需要重新运行 FeatureEngine，feature 层才会反映最新数据。